

Умножение матриц

1. Работа с двумерными массивами

1.1. Объявить внутри функции `main(...)` три указателя типа `double**`, которые далее будут использованы для хранения матриц A, B и C.

1.2. Написать функцию `AllocateMemory`:

тип результата: `void`,

аргументы: указатель на матрицу и размер матрицы.

Функция осуществляет резервирование необходимого объема памяти для хранения матрицы размером $n \times n$. Данная функция должна быть вызвана трижды:

```
AllocateMemory(A, n); AllocateMemory(B, n); AllocateMemory(C, n);
```

где n – произвольно задаваемый в функции `main` размер матрицы.

1.3. Написать функцию `FillAB`:

тип результата: `void`,

аргументы: указатели на матрицы A и B и размер матрицы.

Функция производит заполнение указанных матриц по правилу:

```
Ai,j = cos(i - j); Bi,j = 10.0 / (i - j + 5.0);
```

1.4. Написать функцию `FillZero`:

тип результата: `void`,

аргументы: указатель на матрицу и размер матрицы

Функция производит заполнение матрицы нулями.

1.5. Написать функцию `MultiplyIJK`:

тип результата: `double`,

аргументы: указатели на матрицы A, B, C и размер матрицы.

Функция производит вычисление произведения матриц $C = A \cdot B$ и возвращает время, потраченное на выполнение умножения матриц.

Псевдокод алгоритма:

```
for i:
    for j:
        for k:
            Ci,j += ...
```

Чтобы засечь время, можно использовать следующий механизм:

```
#include <chrono>
auto start = std::chrono::steady_clock::now();
    <some operation>
auto finish = std::chrono::steady_clock::now();
std::chrono::duration<double> time = finish - start;
std::cout << "Time = " << time.count() << " seconds";
```

1.6. Написать функцию `ReleaseMemory`:

тип результата: `void`,

аргументы: указатель на матрицу и размер матрицы.

Функция осуществляет освобождение ранее зарезервированной под хранение матрицы памяти

Вычислительный эксперимент:

Вызвать из `main` вышеперечисленные функции в порядке 1.2–1.6; вывести на экран время, которое потребовалось на перемножение матриц, элемент матрицы $C_{n/2, n/2}$, а также оценку производительности компьютера во флопах (число операций умножения и сложения, выполняемых над числами с плавающей точкой, в секунду); на практике удобнее оперировать гигафлопсами (GFlops), для этого результат нужно поделить на 2^{30} . Сравнить результаты в режимах компиляции `Debug` и `Release`.

(Производительность одного ядра современного процессора обычно лежит в диапазоне 10...100 GFlops; хорошим результатом нужно признать достижение 30...50 % от нее)

2. Оптимизация алгоритма – I

Модифицировать алгоритм (создать вторую программу!), отказавшись от указателей на матрицы вида `double**` в пользу указателей вида `double*`, и хранить матрицы в виде одномерных массивов длиной n^2 по строкам. Оценить изменения в производительности, если они будут наблюдаться.

(Результаты могут быть различными для различных компьютеров, компиляторов и режимов компиляции Debug/Release)

3. Оптимизация алгоритма – II

Написать функции `MultiplyJKI`, `MultiplyKIJ`, `MultiplyIKJ`, `MultiplyJIK`, `MultiplyKJI`, аналогичные указанной в п. 1.5, и различающиеся только порядком следования циклов. Модифицировать `main`, вызвав последовательно все 6 вариантов алгоритма перемножения (не забывая перед каждым вызовом обнулять матрицу `C`), убедиться в правильности результата на примере среднего элемента матрицы и проанализировать время, требуемое на перемножение матриц в разных алгоритмах (и, соответственно, производительность).

Проверить полученные тенденции на «устойчивость», произведя вычисления для матриц большего и меньшего размера (к примеру, 2048 и 512). При этом полезно предварительно оценить теоретически, во сколько раз больше или меньше операций будет выполняться при перемножении матриц вдвое большего или меньшего размера и проверить, насколько дольше фактически выполняется алгоритм.

4. Работа с транспонированными матрицами

Модифицировать ранее написанные алгоритмы для перемножения матриц с транспонированием одной из них: $C = A^T \cdot B$ или $C = A \cdot B^T$ (создать на основе ранее написанных новую программу!).

При этом транспонированную матрицу в явном виде вычислять и хранить не нужно, факт ее транспонирования следует учесть путем перестановки индексов при обращении к элементам матрицы.

Анализ и выводы

Постараться объяснить наблюдаемые эффекты, в смысле полученных оценок производительности процессора и на их основе попытаться сформулировать правила обработки массивов данных, следование которым позволяет повысить эффективность программ